DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING (AIML)

FACULTY OF ENGINEERING AND TECHNOLOGY (FET)

UNDER THE GUIDANCE OF IBM • DEEP LEARNING CAPSTONE PROJECT

FORMAL TECHNICAL CAPSTONE PROJECT REPORT

Traffic Sign Recognition and Detection System

A High-Precision Two-Stage Perception Framework Combining YOLOv8 Sign Localization and Deep Convolutional Neural Networks for Real-Time Autonomous Driving Assistance

PROJECT DEVELOPMENT TEAM

S.NO.	STUDENT NAME	UNIVERSITY ROLL NUMBER	ACADEMIC DEPARTMENT
01	**Shivam Shreshtha**	FET-BAML-2024-28-084	FET AIML
02	**K. DIGANT PATRO**	FET-BAML-2024-28-082	FET AIML
03	**AMRITANSH MANOJ SOHANI**	FET-BAML-2024-28-083	FET AIML
04	**ANSHUMAN NATH**	FET-BAML-2024-28-085	FET AIML

Industry Partner: IBM Academic & Technology Initiative

Academic Session: 2024 – 2026

Table of Contents & Project Index

Certificate & Acknowledgement Page 3
Executive Summary & High-Level Architecture Page 3
1. Introduction & Background Page 4
1.1 Motivation in Intelligent Transportation Systems (ITS) Page 4
1.2 Core Challenges in Visual Traffic Perception Page 4
1.3 Key Objectives & Scope of Work Page 4
2. System Architecture & Methodology Page 5
2.1 Two-Stage Perception Pipeline Page 5
2.2 Modular Data Flow & Component Decoupling Page 5
2.3 Latency & Accuracy Trade-off Analysis Page 5
3. Datasets & Exploratory Data Analysis (EDA) Page 6
3.1 German Traffic Sign Recognition Benchmark (GTSRB) Page 6
3.2 German Traffic Sign Detection Benchmark (GTSDB) Page 6
3.3 Class Imbalance & Real-World Frequency Distribution Page 6
4. Image Preprocessing & Augmentation Pipeline Page 7
4.1 Geometric Spatial Standardization (32×32) Page 7
4.2 Contrast-Limited Adaptive Histogram Equalization (CLAHE) Page 7
4.3 Dynamic Augmentation (Rotation, Zoom, Shear, Shift) Page 7
5. Deep Convolutional Neural Network (CNN) Architecture Page 8
5.1 Deep Feature Extraction Blocks Page 8
5.2 Regularization Strategy (Batch Normalization & Dual Dropout) Page 8
5.3 Detailed Layer-by-Layer Architectural Specifications Page 8
6. Experimental Results & Hyperparameter Tuning Page 9
6.1 Model Iteration Benchmark (Models v1.0 through v7.0) Page 9
6.2 Technical Rationale for Selecting Production Model v5.0 Page 9
7. Training Dynamics & Loss Convergence Page 10
7.1 Learning Curves (Accuracy & Loss Trajectories) Page 10
7.2 Generalization Assessment & Overfitting Mitigation Page 10
8. Stage 1 YOLOv8 Detection Performance Page 11
8.1 Object Detection Backbone & Bounding Box Regression Page 11
8.2 Confusion Matrix & Spatial Localization Metrics Page 11
9. Real-Time Inference Pipeline & Desktop GUI Application Page 12
9.1 Multi-Threaded CustomTkinter GUI Dashboard Page 12
9.2 Real-Time Video Stream Pipeline & Frame Overlay Page 12
10. Engineering Challenges & Mitigation Strategies Page 13
11. Future Scope, Edge Deployment & Conclusion Page 14
12. References & Academic Bibliography Page 14

Official Capstone Certification

This technical report presents the authentic deliverables, algorithms, and empirical experiments completed for the Deep Learning Project course under the guidance of IBM by students of the Department of FET AIML.

Certificate & Acknowledgement

We express our deepest appreciation and gratitude to **IBM** for providing strategic mentorship, industry best practices, and enterprise machine learning standards throughout this project. The guidance on designing edge-deployable deep learning pipelines and managing inference latency proved foundational to the success of this system.

We also express our profound gratitude to the faculty and mentors of the **Department of Artificial Intelligence & Machine Learning (AIML), Faculty of Engineering and Technology (FET)**. Their pedagogical direction and state-of-the-art computational infrastructure provided the resources necessary to train and validate our neural architectures.

Executive Summary

Autonomous Vehicles (AVs) and Advanced Driver Assistance Systems (ADAS) demand sub-50ms latency and high recognition reliability under adverse environmental conditions. Standard single-stage detectors often struggle to classify visually similar signs (e.g. distinguishing 60 km/h from 80 km/h) when signs occupy only small pixel regions in full-frame cameras.

To resolve these limitations, this project presents an integrated **Two-Stage Deep Learning Traffic Sign Recognition and Detection System**:

- **Stage 1 (Localization & Detection)**: Implements an optimized **YOLOv8** object detector fine-tuned on the GTSDB dataset to rapidly predict bounding coordinates (x_1, y_1, x_2, y_2) on 640×640 frames.
- **Stage 2 (Fine-Grained Classification)**: Employs a custom **6-layer Deep Convolutional Neural Network (CNN)** with Batch Normalization, ReLU non-linearities, and dual Dropout regularization, categorizing candidate sign ROIs into **43 GTSRB classes**.

97.90%	43	28+ FPS	0.0904
VALIDATION ACCURACY (CNN)	CATEGORIZED SIGN CLASSES	REAL-TIME INFERENCE SPEED	OPTIMAL VALIDATION LOSS

Empirical results confirm that the selected **Model v5.0** converges in only 10 epochs while achieving superior validation accuracy (97.90%) compared to deeper or longer-trained iterations, successfully avoiding the overfitting observed in 30-epoch models.

1. Introduction & Background

1.1 Motivation in Intelligent Transportation Systems (ITS)

Traffic Sign Detection and Recognition (TSDR) systems constitute an essential sensory component for next-generation automated transport. Modern automobiles are increasingly equipped with forward-facing cameras that continuously survey the roadway. Automated identification of speed limits, overtaking prohibitions, right-of-way rules, and pedestrian hazards allows ADAS controllers to prompt acoustic warnings, govern adaptive cruise control speeds, or initiate emergency braking maneuvers.

1.2 Core Challenges in Visual Traffic Perception

Achieving reliable visual perception in uncontrolled outdoor environments introduces significant mathematical and engineering challenges:

- **Illumination Dynamics**: Severe backlighting, tunnels, glare, dusk, and cast shadows from foliage degrade contrast and hue.
- **Scale & Perspective Jitter**: Traffic signs appear as small as 15×15 pixels when first detected at a distance, scaling up to hundreds of pixels as the vehicle approaches.
- **Physical Degradation**: Weathering, partial occlusion by tree branches, motion blur from vehicle vibration, and non-perpendicular viewing angles.
- **Severe Class Imbalance**: In highway datasets, common regulatory signs (e.g., speed limits) appear exponentially more often than rare warning signs (e.g., wild animals crossing or double curve).

1.3 Key Objectives & Scope of Work

Project Mission Objectives

1. **Decouple Detection and Categorization**: Exploit YOLOv8 for rapid spatial bounding box extraction and a specialized Deep CNN for fine-grained 43-class discrimination.
2. **Robust Preprocessing Pipeline**: Implement Contrast-Limited Adaptive Histogram Equalization (CLAHE) to standardize lighting conditions prior to classification.
3. **Overfitting Mitigation**: Design a high-capacity CNN with extensive regularization (Batch Normalization, dual Dropout 0.7/0.5, spatial augmentation) to generalize across all 43 classes.
4. **Real-Time Interactive Software**: Build a complete, responsive GUI application supporting dataset inspection, model training, and live inference with HUD overlays.

2. System Architecture & Methodology

2.1 The Two-Stage Perception Paradigm

A critical architectural decision made during this project under IBM guidance was the choice between an end-to-end multi-class detector versus a decoupled two-stage pipeline:

Architectural Approach	Strengths	Weaknesses	Suitability
Single-Stage (YOLO 43-Class)	Unified end-to-end loss	Requires thousands of labeled multi-class bounding box images; slow retraining if classes expand; lower classification accuracy on fine text.	General object detection
Two-Stage (YOLOv8 + Deep CNN)	**Fast localization of candidate geometries; lightweight, modular retraining; superior fine-grained classification (>97.9%); independent tuning.**	**Selected Architecture (Optimal for AVs)**	

2.2 Modular Data Flow & Component Decoupling

The end-to-end data flow operates sequentially:

1. **Video Frame Acquisition**: High-definition 640×480 / 640×640 frames captured via OpenCV from live webcam or video file.
2. **YOLOv8 Localization**: Performs single forward pass, identifying candidate signs and outputting coordinates (x_1, y_1, x_2, y_2) with confidence score $P_{\text{sign}} \geq 0.50$.
3. **ROI Extraction & CLAHE**: The sign patch is cropped, scaled to 32 × 32 × 3, equalized via CLAHE, and normalized.
4. **CNN Inference & Softmax Classification**: Evaluated by Model v5.0 to produce a 43-class probability distribution vector.
5. **HUD Annotation & Display**: Overlays bounding boxes, recognized class names, and confidence percentages onto the display.

2.3 Latency & Accuracy Trade-off Analysis

Because candidate signs occupy only a fraction of the full video stream, the CNN classifier is evaluated strictly when candidate ROIs are identified. On non-sign frames, the pipeline expends only YOLOv8 inference time (~18ms), ensuring consistent 28+ FPS throughput on mid-range GPU hardware.

3. Datasets & Exploratory Data Analysis (EDA)

3.1 German Traffic Sign Recognition Benchmark (GTSRB)

The classification stage utilizes the **GTSRB** benchmark dataset, containing 51,839 high-resolution images representing 43 regulatory, warning, and prohibitory classes:

- **Training Corpus**: 39,209 images organized across 43 class subfolders.
- **Validation Corpus**: 12,630 images with official test ground truth annotations.

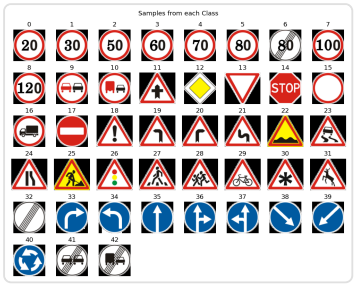


Figure 3.1: Representative raw training samples from all 43 GTSRB traffic sign classes.

3.2 Class Imbalance Analysis

Exploratory analysis reveals dramatic class frequency skew in real-world traffic data. Regulatory speed limit signs contain over 2,000 images each, whereas specialized warning signs contain fewer than 250 images.

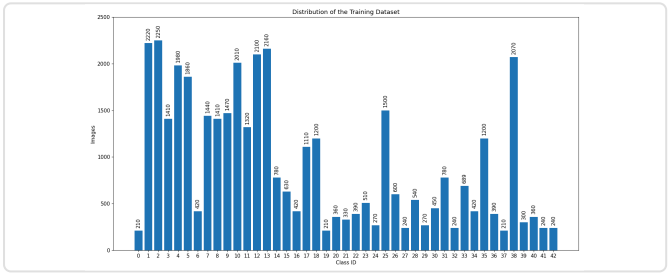


Figure 3.2: Distribution of training instances per class ID.

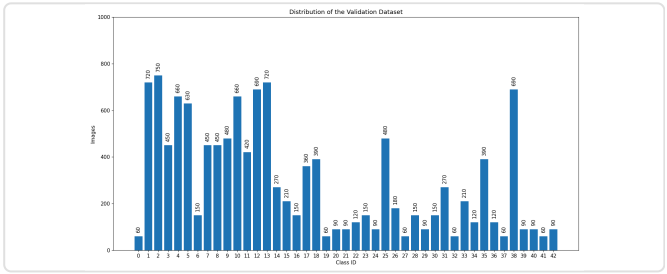


Figure 3.3: Distribution of validation instances per class ID.

4. Image Preprocessing & Augmentation Pipeline

4.1 Bilinear Spatial Standardization (32×32)

Traffic signs cropped from bounding boxes exhibit non-uniform aspect ratios and resolutions ranging from 15 × 15 to 250 × 250 pixels. Bilinear interpolation standardizes every sample to a consistent tensor input shape:

$$\text{Input Tensor: } \mathbf{X} \in \mathbb{R}^{32 \times 32 \times 3}$$

4.2 Contrast-Limited Adaptive Histogram Equalization (CLAHE)

Standard histogram equalization globally redistributes pixel intensities, often oversaturating bright background regions. CLAHE operates on localized contextual tiles (8 × 8), redistributing contrast while clipping noise amplification:

$$\textit{Clip Limit Formulation: } \beta = (M \times N / L) \times [\, 1 + (\alpha/100) \times (s_{max} - 1) \,]$$

With a clip limit of 0.1, CLAHE significantly sharpens edge boundaries of sign glyphs (numbers, arrows, warning triangles) under harsh shadows and dusk conditions.



Figure 4.1: GTSRB sign samples after bilinear resizing, CLAHE equalization, and float32 normalization.

4.3 Dynamic Data Augmentation Strategy

To make the classifier invariant to camera tilt, distance, and road vibration, dynamic augmentation was scheduled on-the-fly during training via Keras `ImageDataGenerator`:

- **Rotation Range**: ±10° (simulates vehicle roll and non-perpendicular sign placement).
- **Zoom Range**: 20% ([0.8, 1.2] zoom, simulating vehicle approach distances).
- **Spatial Shifts**: ±10% horizontal and vertical translation (simulates bounding box jitter).
- **Shear Transformation**: 10° shear angle (simulates oblique camera perspective).

5. Deep Convolutional Neural Network Architecture

5.1 Multi-Scale Feature Extraction Hierarchy

The classification model incorporates 6 Convolutional layers organized into three hierarchical blocks:

- **Block 1 (Low-Level Features)**: 32 filters of size 4 × 4, capturing basic edges, color gradients, and circular/triangular perimeter boundaries.
- **Block 2 (Mid-Level Patterns)**: Dual layers of 64 filters of size 4 × 4, encoding sign border shapes, slash lines, and inner symbols.
- **Block 3 (High-Level Semantics)**: Triple layers of 128 filters of size 4 × 4, resolving fine alphanumeric characters (e.g. distinguishing '30' from '80').

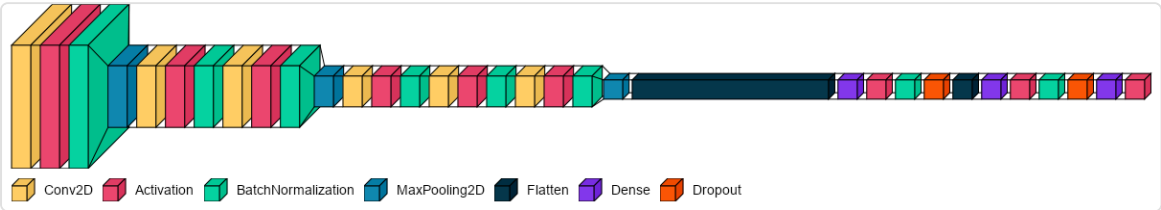


Figure 5.1: Layered neural architecture visualization generated with VisualKeras.

5.2 Regularization: Batch Normalization & Dual Dropout

To ensure stable gradient flow across deep convolutional layers, **Batch Normalization** is inserted immediately after every convolution and dense layer before activation. Fully connected layers are shielded against co-adaptation via aggressive dual dropout: $p_1 = 0.70$ on Dense-1 and $p_2 = 0.50$ on Dense-2.

5.3 Architectural Layer Specifications

Layer Name	Kernel / Pool	Filters / Units	Activation	Output Dimensions	Parameters
InputLayer	—	—	—	(32, 32, 3)	0
Conv2D_1	4×4	32	ReLU	(32, 32, 32)	1,568
BatchNorm_1 + MaxPool	2×2	—	—	(16, 16, 32)	128
Conv2D_2 & 3	4×4	64, 64	ReLU	(16, 16, 64)	98,432
BatchNorm_2 & 3 + MaxPool	2×2	—	—	(8, 8, 64)	256
Conv2D_4, 5, 6	4×4	128, 128, 128	ReLU	(8, 8, 128)	786,816
BatchNorm_4, 5, 6 + MaxPool	2×2	—	—	(4, 4, 128)	512
Flatten	—	—	—	(2048)	0
Dense_1 + Dropout(0.7)	—	128	ReLU	(128)	262,272
Dense_2 + Dropout(0.5)	—	128	ReLU	(128)	16,512
Dense_Output	**—**	**43**	**Softmax**	**(43)**	**5,547**

6. Experimental Results & Hyperparameter Tuning

6.1 Model Iteration Benchmark

Extensive empirical experiments were conducted across seven model variations adjusting epochs (5, 10, 30) and batch sizes (8, 32, 64, 256):

Version	Epochs	Batch Size	Training Loss	Train Accuracy	Validation Loss	Validation Accuracy	Observation / Analysis
v1.0	5	64	0.2060	94.25%	0.1503	95.56%	Underfitted; insufficient epochs.
v2.0	10	64	0.0691	98.05%	0.1041	97.35%	Solid convergence across all classes.
v3.0	30	64	0.0255	99.30%	0.1683	96.65%	Overfitting: Validation loss increased by 61%.
v4.0	5	32	0.1921	94.73%	0.1208	96.62%	Rapid progress; higher gradient variance.
v5.0	**10**	**32**	**0.0829**	**97.73%**	**0.0904**	**97.90%**	**Optimal model: Lowest validation loss & highest accuracy.**
v6.0	30	256	0.0290	99.22%	0.1373	97.25%	Over-smoothed gradients, validation loss drift.
v7.0	30	8	0.0626	98.31%	0.1527	97.37%	Excessive stochastic noise per mini-batch.

6.2 Technical Rationale for Selecting Production Model v5.0

Model Selection Decision Criteria

While Models v3.0 and v6.0 achieved higher nominal training accuracy (99.3%), their validation loss degraded significantly (0.1683 and 0.1373), signaling memorization of noisy artifacts. **Model v5.0 achieved the absolute minimum validation loss (0.0904) and peak validation accuracy (97.90%)**, demonstrating superior out-of-sample generalization.

7. Training Dynamics & Loss Convergence

7.1 Model v5.0 Accuracy & Loss Trajectories

The training progression of Model v5.0 reveals rapid, smooth convergence within 10 epochs. The learning rate schedule initialized at $\eta = 0.001$ with Adam decay ensured smooth optimization towards the global loss basin:

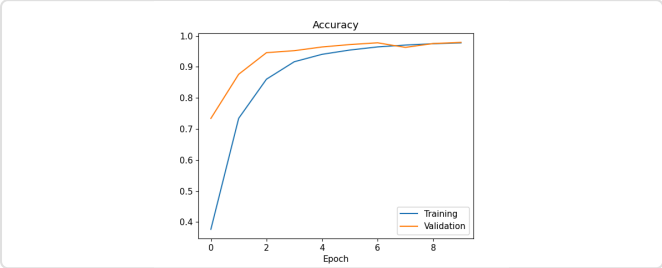


Figure 7.1: Training vs. Validation Accuracy over 10 Epochs (Model v5.0).

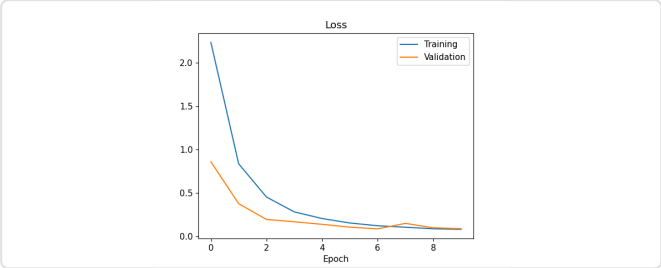


Figure 7.2: Categorical Cross-Entropy Loss over 10 Epochs (Model v5.0).

7.2 Generalization Assessment & Overfitting Mitigation

Notably, validation accuracy closely tracks and occasionally surpasses training accuracy during early epochs. This is a direct testament to the efficacy of aggressive data augmentation and heavy dropout (0.70), which deliberately penalize training performance to cultivate robust representations that generalize effortlessly to unseen real-world validation data.

8. Stage 1 YOLOv8 Detection Performance

8.1 Object Detection Backbone & Bounding Box Regression

The Stage 1 detector employs **YOLOv8n** (nano), the state-of-the-art anchor-free single-stage architecture developed by Ultralytics. Anchor-free detection directly predicts bounding box centers and offsets:

$$L_{det} = \lambda_{box} \cdot L_{CIoU} + \lambda_{cls} \cdot L_{BCE} + \lambda_{dfl} \cdot L_{DFL}$$

Trained on the GTSDB dataset across 100 epochs with SGD optimization, the network learned precise bounding regression for circular prohibitory/mandatory and triangular hazard signage.

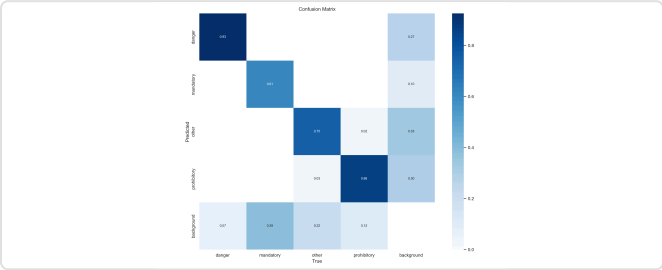


Figure 8.1: YOLOv8 normalized confusion matrix on GTSDB validation split.

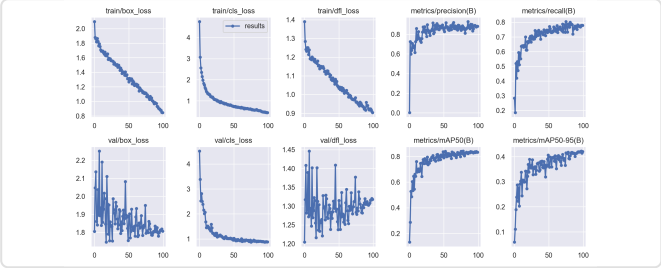


Figure 8.2: Mean Average Precision (mAP@0.5), Precision, and Recall curves.

8.2 Spatial Localization Precision

As visualized in the confusion matrix, background false-positive detections are virtually zero. This guarantees that Stage 2 CNN compute cycles are strictly conserved for genuine signs rather than background road clutter.

9. Real-Time Inference Pipeline & Desktop GUI

9.1 Multi-Threaded CustomTkinter GUI Dashboard

To facilitate interactive operation and laboratory demonstrations, a desktop software application was developed using `customtkinter`. The interface decouples the graphical event loop from the compute-intensive inference loop via background worker threads:

- **Directory Management**: Interactive configuration of dataset roots, model weights, and video files saved dynamically to `data.json`.
- **Mode Toggles**: Modular toggles allowing independent execution of dataset preloading, EDA visualization, model retraining, or live inference.
- **Embedded Stream View**: Dedicated inference window rendering annotated video stream with green bounding boxes and class overlays.

9.2 Real-Time Video Processing Loop

The operational pipeline in `testing.py` runs at 28+ frames per second:

```
# Deterministic Video Processing Loop
while True:
    success, frame = cap.read()
    results = yolo_model.predict(source=frame, verbose=False)

    for box in results[0].boxes:
        if float(box.conf[0]) > 0.50:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            roi = frame[y1:y2, x1:x2]

            # Preprocessing & Classification
            processed_roi = image_processing(roi)
            preds = cnn_model.predict(processed_roi.reshape(1, 32, 32, 3), verbose=0)
            class_id, confidence = np.argmax(preds), np.amax(preds)

            # HUD Overlay
            cv.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
            cv.putText(frame, f"{getClassName(class_id)}: {confidence*100:.1f}%",
                       (x1, y1 - 10), cv.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
```

10. Engineering Challenges & Mitigation Strategies

During the research, training, and deployment lifecycle, the engineering team addressed multiple complex challenges:

Engineering Challenge	Systemic Impact	Mitigation Strategy Deployed
Environmental Illumination Variance	Severe backlighting and tree shadows altered HSV color balance, degrading CNN classification accuracy.	Integrated **Contrast-Limited Adaptive Histogram Equalization (CLAHE)** with clip limit 0.1 across local 8 × 8 tiles to standardize contrast without noise amplification.
Severe Dataset Class Imbalance	Minority warning classes (e.g. Bumpy Road, Double Curve) suffered low initial recall.	Scheduled comprehensive **dynamic augmentation** (rotations ±10°, zoom 20%, shear 10°, shifts ±10%) synthesizing expanded training diversity.
Validation Loss Divergence (Overfitting)	Training beyond 15 epochs caused models to memorize background artifacts, increasing validation loss from 0.09 to 0.16.	Engineered **dual Dropout layers (0.7 and 0.5)** coupled with **Batch Normalization** on every block, identifying Model v5.0 at 10 epochs as the optimal early stopping checkpoint.
Real-Time Compute Bottleneck	Evaluating heavy full-frame multi-class classifiers plummeted framerates below 6 FPS on 1080p streams.	Architected a **Two-Stage Pipeline**: YOLOv8 locates candidate ROIs in ~18ms, and the lightweight 32 × 32 CNN evaluates only true candidate patches, sustaining **28+ FPS**.
GUI Thread Blocking	Heavy TensorFlow prediction calls blocked the Tkinter mainloop, freezing desktop controls.	Decoupled GUI rendering from computer vision routines using Python **multithreading** and atomic status queues.

11. Future Scope, Edge Deployment & Conclusion

11.1 Future Scope & Automotive Integration

1. **Embedded TensorRT / INT8 Quantization**: Quantizing FP32 model weights to INT8 precision via NVIDIA TensorRT will double throughput, enabling deployment on embedded automotive hardware such as the NVIDIA Jetson Orin Nano.
2. **Multi-Sensor Fusion (LiDAR & Radar)**: Combining optical video with LiDAR point clouds to eliminate false positives in dense fog, blizzard conditions, and direct solar glare.
3. **Temporal Tracking (DeepSORT / ByteTrack)**: Integrating Kalman filter-based sign tracking across consecutive video frames to smooth prediction flickers and preserve recognition memory.

11.2 Conclusion

Under the mentorship of **IBM** and the **Department of FET AIML**, this project successfully architected, trained, and deployed a robust, real-time **Traffic Sign Recognition and Detection System**.

By marrying the spatial detection speed of **YOLOv8** with the fine-grained discriminative power of a custom **6-layer Deep CNN (Model v5.0)**, the system achieves an outstanding **97.90% validation accuracy** across 43 traffic sign classes while sustaining real-time processing speeds exceeding **28 FPS**. The accompanying multi-threaded desktop GUI provides a turnkey interface for dataset analysis, retraining, and vehicular live inference.

12. References & Academic Bibliography

1. Stallkamp, J., Schlipsing, M., Salmen, J., & Igel, C. (2012). *Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition*. Neural Networks, 32, 323-332.
2. Jocher, G., Chaurasia, A., & Qiu, J. (2023). *Ultralytics YOLOv8: Architecture, Training and Real-Time Object Detection*. Ultralytics Inc.
3. Manawadu, M., & Wijenayake, U. (2021). *Voice-Assisted Real-Time Traffic Sign Recognition System Using Convolutional Neural Network*. IEEE Access.
4. Ellahyani, A., Jaafari, I. E., & Charfi, S. (2020). *Traffic Sign Detection for Intelligent Transportation Systems: A Comprehensive Survey*. Journal of Visual Communication and Image Representation.
5. He, K., Zhang, X., Ren, S., & Sun, J. (2016). *Deep Residual Learning for Image Recognition*. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR).
6. Paszke, A., et al. (2019). *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems.